Streszczenie

TODO

Abstract

TODO

Spis treści

1 Wstęp 3

2 Notacja Ficha i jej interpretacja 5

3 Ustawienie modelu 7

4 Parsowanie dowodów w notacji Ficha 8

5 Generowanie korpusu 9

6 Trenowanie i kożystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYSTKIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!) 9

7 3 strategie 9

8 Rozbijanie problemów na podproblemy 11

9 Klasa `node` 13

1 Wstęp

Dedukcja naturalna jest metodą dowodzenia twierdzń logicznych przez wychodzenie z pewnych założeń i modyfikowanie ich za pomocą ściśle zdefiniowanych reguł wnioskowania.

Definicja (sekwent)

Sekwentem nazywamy wyrażenie postaci

$$\Gamma \vdash \varphi$$

gdzie Γ jest skończonym zbiorem formuł (zapisywanych po przecinku i bez nawiasów klamrowych) zwanym *kontekstem*, a φ jest formułą zwaną *tezą sekwentu*. Sekwent $\Gamma \vdash \varphi$ interpretujemy jako stwierdzenie, że jeśli założymy, że wszystkie formuły z Γ są prawdziwe, da się dowieść φ. W rachunku zdań w logice klasycznej, który będzie jedyną logiką w której się będziemy poruszać, zdanie prawdziwe we wszystkich wartościowaniach jest dowodliwe, tak więc $\Gamma \vdash \phi$ jest równoważne stwierdzeniu jeżeli wszystkie formuły kontekstu Γ są prawdziwe to φ jest prawdziwe.

Definicja (reguła wnioskowania)

Regułą wnioskowania nazywamy schemat postaci

$$\frac{S_1 \qquad S_2 \qquad \dots \qquad S_n}{S}(R)$$

gdzie $S_1, \dots, S_n, S$ są sekwentami. Sekwenty $S_1, \dots, S_n$ nazywamy *przesłankami reguły*, natomiast sekwent S nazywamy *konkluzją reguły*. R to z kolei *indeks reguły*, który służy do jej identyfikacji. Jeżeli przesłanki reguły są prawdziwe to z definicji jej konkluzja jest również prawdziwa.

Dowody w dedukcji naturalnej można utożsamiać z drzewami, których węzłami są sekwenty, a krawędzie są indeksowane indeksami reguł wnioskowania.

W nieniejszej pracy skupimy się na dowodzeniu twierdzeń w logice klasycznej z następującymi spójnikiami logicznymi: koniunkcją ($\wedge$), alternatywą ($\vee$), negacją ($\neg$) oraz implikacją ($\rightarrow$). regułami wnioskowania:

$$\frac{}{\varphi \vdash \varphi}\ (\text{Ass}) \qquad \frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi}\ (\text{W})$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \to \psi}\ (\to I) \qquad \frac{\Gamma_1 \vdash \varphi \to \psi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi}\ (\to E)$$

$$\frac{\Gamma, \varphi \vdash \bot}{\Gamma \vdash \neg\varphi}\ (\neg I) \qquad \frac{\Gamma_1 \vdash \neg\varphi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \bot}\ (\neg E)$$

$$\frac{\Gamma_1 \vdash \varphi \qquad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi \wedge \psi}\ (\wedge I) \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi}\ (\wedge E_1)$$

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\ (\vee I_1) \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi}\ (\wedge E_2)$$

$$\frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi}\ (\vee I_2) \qquad \frac{\Gamma_1 \vdash \varphi \vee \psi \qquad \Gamma_2, \varphi \vdash \rho \qquad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho}\ (\vee E)$$

$$\frac{}{\vdash \top}\ (\top I) \qquad \frac{\Gamma \vdash \bot}{\Gamma \vdash \varphi}\ (\bot E)$$

$$\frac{\Gamma_1, \varphi \vdash \psi \qquad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi}\ (\leftrightarrow I) \qquad \frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \qquad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi}\ (\leftrightarrow E_1)$$

$$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \qquad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi}\ (\leftrightarrow E_2) \qquad \frac{\Gamma, \neg\varphi \vdash \bot}{\Gamma \vdash \varphi}\ (\text{RAA})$$

$$\frac{\Gamma \vdash \neg\neg\varphi}{\Gamma \vdash \varphi}\ (\neg\neg) \qquad \frac{}{\vdash \varphi \vee \neg\varphi}\ (\text{TND})$$

Tabela 1: Reguły wnioskowania rozważanego systemu

Oto przykładowy dowód w dedukcji naturalnej w notacji drzewiastej:

$$\frac{\dfrac{\dfrac{}{a \vee b \vdash a \vee b}\ (\text{Ass}) \qquad \dfrac{\dfrac{}{a \vdash a}\ (\text{Ass})}{a \vdash b \vee a}\ (\vee I_2) \qquad \dfrac{\dfrac{}{b \vdash b}\ (\text{Ass})}{b \vdash b \vee a}\ (\vee I_1)}{a \vee b \vdash b \vee a}\ (\vee E)}{\vdash (a \vee b) \to (b \vee a)}\ (\to I)$$

Dużą zaletą dedukcji naturalnej jest to, że niejako symuluje ona sposób, w jaki ludzie przeprowadzają wnioskowanie, zarówno w matematyce, jak i w codziennym życiu. Dowody mają swoją drzweiastą strukturę, którą można łatwo analizować. Inną zaletą tej metody jest jej uniwersalność. Dodając odpowiednie reguły wnioskowania możemy otrzymać dedukcję naturalną dla różnych systemów logicznych np logiki pierwszego i drugiego rzędu, czy logiki modalnej. Jeszcze jedną zaletą jest to, że w przypadku większości użytecznych dowodów jesteśmy w stanie uniknąć wykładniczo dużej złożoności wielu innych systemów dowodzenia sprowadzających się do rozpatrzenia wszystkich możliwych przypadków. Z drugiej strony dedukcja naturalna jest trudna w zaitomatyzowaniu. Zwykle dowód przeprowadzamy z góry na dół, zaczynając od założeń i kończąc na tezie. Jednak nie zawsze jasne jest, którą regułę wnioskowania zastosować i co gorsza wyniki zastosowania niektórych reguł mogą zawierać podformuły których nie ma w przesłankach, lub jej wybór nie jest jednoznaczny. Np używając reguły

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\ (\vee I_1)$$

w przesłance nie mamy informacji o formule ψ, która pojawi się w konkluzji. Jednak podobieństwo dedukcji naturalnej do ludzkiego sposobu wnioskowania sprawia, oraz pewien dający się zauważyć porządek w zapisie dowodów sprawia, że gdzie klasyczna implementacja dedukcji naturalnej jest trudna, wydaje się ona dobrze nadawać do implementacji z użyciem metod sztucznej inteligencji, w tym modeli językowych o arhirtekturze transformerów.

W niniejszej pracy przedstawimy implementację automatycznego systemu dowodzenia twierdzeń w logice klasycznej przy użyciu odpowiednio wytrenowanego modelu nano-GPT oraz przedstawimy wyniki jego działania i jak dobrze radzi sobie z przeprowadzaniem dowodów. Konstrukcja systemu

będzie opisana krok po kroku, zaczynając od zaprogrogramowania logicznej struktury systemu, tłumaczenia zapisu dowodu w formie tekstowej na strukturę logiczną i sprawdzanie poprawności dowodu, generacja korpusu treningowego, trenowanie modelu oraz kolejne etapy konstrukcj samego systemu dowodzenia(wybór możliwych linii, równoległość prowadzenia kilku ścieżek rozumowania, itp).

2 Notacja Ficha i jej interpretacja

Notacja Ficha jest sposobem zapisu z dowodów dedukcji naturalnej w formie tekstowej. Jest on wygodny, ponieważ klasyczna metoda zapisywania drzew dowodu potrafi zajmować bardzo dużo miejsca i trudno użyć do niej modeli językowych. Notacja Ficha jest z kolei metodą zapisu dowodu w formie tekstu, gdzie każda linia w specyficzny sposób koduje sekwent uzyskany przez używanie reguł do sekwentów kodowanych w poprzednich liniach.

Przyjmijmy, następującą funkcję:

$$\texttt{Fitch(i)} := \text{sekwent kodowany przez linię o indeksie } \texttt{i}$$

oraz oznaczenie

$$\texttt{s.conclusion} := \text{teza sekwentu } \texttt{s}$$

$$\texttt{s.assumptions} := \text{kontekst sekwentu } \texttt{s}$$

Linia dowodu w notacji Ficha można podzielić na 4 części: indeks linii, konkluzja sekwentu kodowanego przez tą linię, nazwa reguły wnioskowania i indeksy linii które używamy przy użyciu tej reguły.

Poniżej przedstawimy interpretację poszczególnych reguł wnioskowania w notacji Ficha, a także ich odpowiedniki w notacji drzewiastej.

Notacja Fitcha	**Notacja drzewiasta**
i. ϕ assumption	$\dfrac{}{\phi \vdash \phi}\ (\text{Ass})$
i. ϕ weakening, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (W)$
j. ϕ ... ... k. ψ ... ... i. $\phi \to \psi$ →-introduction, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\to I)$
j. ϕ ... ... k. $\bot$ ... ... i. $\neg\phi$ ¬-introduction, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\neg I)$
j. ϕ ... ... k. $\phi \to \psi$ ... ... i. ψ →-elimination k, j	$\dfrac{\texttt{Fitch(k)} \quad \texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\to E)$
j. $\neg\phi$ ... ... k. ϕ ... ... i. $\bot$ ¬-elimination, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\neg E)$

j. ϕ ... ... k. ψ ... ... i. $\phi \wedge \psi$ $\wedge$-introduction, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\wedge I)$
j. ϕ ... ... i. $\phi \vee \psi$ $\vee$-introduction1, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\vee I_1)$
j. ψ ... ... i. $\phi \vee \psi$ $\vee$-introduction2, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\vee I_2)$
i. $\top$ $\top$-introduction	$\dfrac{}{\vdash \top}\ (\top I)$
j. $\phi \wedge \psi$ ... ... i. ϕ $\wedge$-elimination1, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\wedge E_1)$
j. $\phi \wedge \psi$ ... ... i. ψ $\wedge$-elimination2, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\wedge E_2)$
j. $\phi \vee \psi$ ... ... k. ρ ... ... l. ρ ... ... i. ρ $\vee$-elimination, j, k, l	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)} \quad \texttt{Fitch(l)}}{\texttt{Fitch(i)}}\ (\vee E)$
j. $\perp$ ... ... i. ϕ $\perp$-elimination, j	$\dfrac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\perp E)$
j. ψ ... ... k. ϕ ... ... i. $\phi \leftrightarrow \psi$ $\leftrightarrow$-introduction, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow I)$
j. $\phi \leftrightarrow \psi$ ... ... k. ϕ ... ... i. ψ $\leftrightarrow$-elimination1, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow E_1)$
j. $\phi \leftrightarrow \psi$ ... ... k. ψ ... ... i. ϕ $\leftrightarrow$-elimination2, j, k	$\dfrac{\texttt{Fitch(j)} \quad \texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\leftrightarrow E_2)$
j. $\neg\phi$ ... ... k. $\perp$ ... ... i. ϕ RAA, j-k	$\dfrac{\texttt{Fitch(k)}}{\texttt{Fitch(i)}}\ (\mathrm{RAA})$

j. $\neg\neg\phi$ ... ... i. ϕ ¬¬-elimination, j	$\frac{\texttt{Fitch(j)}}{\texttt{Fitch(i)}}\ (\neg\neg)$
i. $\phi \vee \neg\phi$ TND	$\frac{}{\vdash \phi \vee \neg\phi}\ (\text{TND})$

Warto zwuważyć, że aby dowody były poprawne muszą być spełnione odpowiednie warunki dotyczące sekwentów do których się odwołujemy, wymuszone przez reguły wnioskowania. Np dla napisu:

j. $\phi \vee \psi$ ...
...
k. ρ ...
...
l. ρ ...
...
i. ρ $\vee$-elimination, j, k, l

musi być spełnione
$\phi \in$ `Fich(k).assumptions` oraz $\psi \in$ `Fich(l).assumptions`.

3 Ustawienie modelu

Bazą naszego systemu są klasy `Relation` o `Variable`. `Relation` reprezentuje relację, zawiera akceptowalne arności, nazwę relacji oraz metodę `toString`, która zwraca jej reprezentację tekstową. W naszym modelu używamy jedynie zmiennych zdaniowych, więc wystarczyłoby jedynie użycie klasy reprezentacej zmienną jednak użycie klasy `Relation` pozwala na łatwe rozszerzenie modelu w przyszłości o predykaty wieloargumentowe. Dla uproszczenia zapisu dodajemy jednak dodajemy fukcję która wczytuje/wypisuje jedynie nazwy zmiennych i zmienną `x` interpretuje jako `'(x)`.

Klasa `Variable` reprezentuje zmienną, którą dajemy jako argument do relacji. Zawiera ona nazwę zmiennej oraz metodę zwracającą jej reprezentację tekstową.

Kolejną klasą jaką definiujemy jest klasa `Formula`, która reprezentuje formułę logiczną. Podstawową i najprostszą formułą jest formuła `Atom`, której implementacja zawiera nazwę relacji którą reprezentuje oraz listę argumentów na których ona zachodzi. Poza formułą atomową mamy również klasy reprezentujące negację, koniunkcję, alternatywę, implikację oraz równoważność. Klasy te zawiera swój typ oraz dwu lub jedno elementową listę podformuł w polu `Interior`.

Kolejną zaimplementowaną klasą jest klasa `Context`, która reprezentuje kontekst dowodzenia, czyli zbiór aktualnie przyjętych formuł. Z powodów które zostaną wyjaśnione w dalszej części pracy `Context` podzieliliśmy, na dwie listy: `base_context` czyli formuły które przyjmujemy za prawdziwe w kontekście całego segmentu dowodu, oraz `additional_context`, czyli formuły, które mogą zmieniać się w każdej linii dowodu. Obiekty klasy `Context` możemy sumować ze sobą jak zbiory metodą `union`, a także odejmować element, lub inny kontekst metodą `remove`. Operacje te wpływają jednak jedynie na `additional_context`, zaś `base_context` pozostaje bez zmian. Aby dwa konteksty można było od siebie odjąć lub dodać, muszą mieć takie samo `base_context`, w przeciwnym wypadku zostaje zwrócony `TypeError("Bad arguments")`.

Sekwent reprezentuje klasa `LittleProblem`, która składa się z kontekstu w polu `assumptions` oraz tezy w polu `conclusion`.

Kolejną klasą jest klasa `Rule`, która reprezentuje regułę wnioskowania. Dziedziczą po niej klasy

- `Assumption`
- `Weakening`
- `ImplicationIntroduction`

- `NegationIntroduction`
- `ImplicationElimination`
- `NegationElimination`
- `ConjunctionIntroduction`
- `DisjunctionIntroduction1`
- `DisjunctionIntroduction2`
- `TruthIntroduction`
- `ConjunctionElimination1`
- `ConjunctionElimination2`
- `DisjunctionElimination`
- `LieElimination`
- `IffIntroduction`
- `IffElimination1`
- `IffElimination2`
- `RAA`
- `NegationOfNegation`
- `TND`

, które reprezentują odpowiednie reguły wnioskowania. Każda z tych klas zawiera metodę `top_down`, która bierze listę sekwentów będącymi przesłankami reguły. Niektóre reguły wymagają doatkowych niejednoznacznych argumentów. Np zgodnie z notacją w Tabeli 1 reguła (Ass) wymaga znajomości formuły ϕ która może być dowolna, z kolei zastosowanie reguły (RAA), wymaga znajomości ϕ, która jest dowolną formułą, której zanegowanie występuje w kontekście sekwentu podanego jako przesłanka. W takim wypadku podajemy te niejednoznaczne formuły, jako dodatkowy argument oprócz przesłanek. Również `base_context` który zakładamy jako prawdziwy dla całego dowodu nie koniecznie da się wydedukować z listy przesłanek (chodzi o sytuacje dla reguł (Ass), (TND) i ($\top I$), gdzie lista przesłanek jest pusta. Wtedy konieczne jest podanie również bazowego kontekstu w formie listy formuł. Oprócz generacji konkluzji reguły `top_down` sprawdza również, czy wszystkie podane argumenty są poprawne. Po pierwsze chodzi o to, czy `base_context` kontekstów przesłanek są takie same, czy listy przesłanek mają odpowiednie długości i czy ich elementy są klasy `LittleProblem`. Kolejną kwestią jest to, czy argumenty, można faktycznie podstawić do danej reguły. Np czy dla reguły $\neg E$ teza pierwszej przesłanki jest negacją tezy drugiej przesłanki, lub czy dla reguły RAA $\neg\phi$ jest w kontekście przesłanki, przy czym ϕ jest przechowywane w argumencie `phi` funkcji `top_down` klasy `RAA`. Jeżeli argumenty nie spełniają odpowiednich warunków, zostaje zwrócony błąd `TypeError("Bad arguments")`.

4 Parsowanie dowodów w notacji Ficha

W celu kontroli poprawności dowodu w notacji Fitcha, jest on w programie parsowany do reprezentującej go struktury logicznej. Dowody reprezentujemy jako słownik, którego kluczami są indeksy linii, a wartościami obiekty klasy `structure_of_line_top_down`. Klasa ta zawiera pola `Key`, które reprezentuje indeks linii, `args_keys`, które reprezentują indeksy linii, które są używane jako przesłanki, `rule`, które reprezentuje regułę wnioskowania, której używamy oraz, `LP`, które reprezentuje sekwent który koduje dana linia. Wartość `LP` jest generowana przez regule `top_down` zastosowaną dla przesłanek reprezentowanych przez `args_keys` i pozostałych argumentów Funkcja `__init__` tej klasy bierze za argument `Key`, `arg_keys`, `rule`, a także `Proof_so_far`, które zawiera sparsowany do formy słownika dotychczasowy dowód, który jest konieczny gdy chcemy odwoływać się do przesłanek, oraz `formula_text`, która

jest wyciągniętą z linii formułą w notacji infiksowej, z której będzie niekiedy konieczne dedukować argumenty nie będące przesłankami w regule `top_down`. Ostatnim argumentem dla funkcji `__init__` jest `BaseCntext`, który jest przyjmowanym w danym dowodzie kontekstem bazowym.

Funkcja `parseProof` tworzy słownik, w którym każdemu indeksowii linii jest przypisany odpowiedni obiekt typu `structure_of_line_top_down`. Funkcja ta jako argumenty przyjmuje dowód w notacji Fitcha w formie tekstowej, oraz używany w dowodzie kontekst bazowy. Funkcja ta wczytuje dowód linia po linii i aktualizuje słownik. Sprawdza również, czy formuła będąca konkluzją pola `LP` obiektu klasy `structure_of_line_top_down` odpowiadająca danej linii jest taka sama jak formuła zapisana w zapisie tekstowej tej linii. Jeżeli tak nie jest oznacza to, że dowód jest niepoprawny i zostaje zwrócony błąd `TypeError("Bad arguments")`.

5 Generowanie korpusu

6 Trenowanie i kożystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYSTKIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!)

7 3 strategie

Celem tej pracy nie jest jedynie skonstruowanie modelu językowego generującego dowody w dedukcji naturalnej, lecz także zbudowanie systemu, który w pewnym stopniu naśladuje sposób, w jaki człowiek dochodzi do poprawnego dowodu. Sam model językowy może generować ciągi tekstu przypominające formalny dowód, jednak nie gwarantuje to ich poprawności logicznej. Z tego powodu każdy wygenerowany krok musi być na bieżąco weryfikowany przy użyciu opisanej wcześniej funkcji `parseProof`.

Drugim problemem jest to, że konstruowanie dowodu rzadko ma charakter całkowicie liniowy. W praktyce często wymaga ono licznych prób, cofania się i wybierania innych ścieżek rozumowania, zanim zostanie znaleziony zapis końcowy. Dlatego druga wersja systemu rozszerza prostą generację kolejnych linii o mechanizm przeszukiwania przestrzeni możliwych kroków dowodowych.

Wreszcie, dla trudniejszych twierdzeń samo lokalne przeszukiwanie kolejnych kroków może okazać się niewystarczające. System może trafiać na ślepe uliczki, ponieważ na początkowym etapie dowodu nie zawsze wiadomo, która strategia doprowadzi do celu. Z tego względu trzecia wersja systemu wprowadza możliwość rozbijania problemu na prostsze podproblemy, które następnie mogą zostać połączone odpowiednią regułą wnioskowania w rozwiązanie problemu wyjściowego.

System był rozwijany etapami. Najprostsza wersja zajmuje się wyłącznie generowaniem kolejnych linii dowodu i sprawdzaniem ich poprawności. Druga wersja dodaje mechanizm przeszukiwania metodą prób i błędów. Ostateczna wersja rozszerza ten schemat o dekompozycję celu na podproblemy. Każda kolejna wersja wykorzystuje poprzednią jako komponent składowy. W dalszej części rozdziału omawiam kolejno wszystkie trzy strategie.

Pierwsza wersja systemu zawiera się w funkcji `silliest_generate_proof`. Jej argumenty to `bundle` będący obiektem klasy `NanoGPTBundle` którym będziemy generować tekst dowodu, `prompt` który jest zmienną tekstową, która koduje twierdzenie, które mamy udowodnić, i ewentualnie początek dowodu, zmienna `temperature` typu `float`, która pozwala kontrolować poziom determinizmu generacji oraz zmienna `max_new_lines` typu `int`, która określa ile linii dowodu chcemy wygenerować i zmienna `max_failed_attempts`, która oznacza ile maksymalnie kolejnych nieudanych prób dopisania linii dowodu akceptujemy. Każdy prompt zaczyna się od słów `Prove that:` a następnie podana jest formuła rachunku zdań w formie infiksowej. Jeżeli kontekst bazowy w generowanym dowodzie ma być niepusty to w drugiej linii znajdują się słowa: `assuming that:` po których wypisana jest pierwsza z formuł bazowego kontekstu. Kolejne formuły są wypisywane po przecinku i każda w nowej linii. Następnie w kolejnych liniach jest dopisywany dowód.

Dowód dopisujemy w pętli, która kończy się kiedy ostatnia linia koduje sekwent, który chcemy udowodnić, liczba nowych linii przekracza `max_new_lines` lub ilość kolejnych nieudanych prób dopisania linii dowodu przekracza `max_failed_attempts`. W każdej iteracji pętli próbujemy dodać jedną linię dowodu. Linia jest odrzucana jeżeli występują w niej zmienne inne niż w docelowym sekwencie, linia

nie spełnia warunków składniowych lub jeśli parsowanie dowodu z nową linią zwraca błąd. Jeżeli linia nie jest odrzucana jest dopisywana do końca dowodu i zerowany jest licznik odrzucanych linii. Funkcja zwraca `prompt_str` z dopisanymi liniami dowodu (linie te nie muszą tworzyć pełnego dowodu tezy, ale intuicyjnym celem tej funjkcji jest po prostu zrobienie pewnego postępu w dowodzie).

Druga wersja systemu jest zaimplementowana w funkcji `find_proof_little_smarter`. Idea tej funkcji sprowadza się do tego, że zamiast tworzyć jeden dowód, tworzymy listę potencjalnych prób dowodu (na początku jest w niej jedynie prompt z zakodowanym problemem do udowodnienia) i w każdej iteracji pętli głównej programu losujemy któryś element tej listy (prawdopodobieństwa są tak ustawione, by im wyższe jest średnie prawdopodobieństwo tokenów w danej próbie dowodu tym wyższe jest prawdopodobieństwo wylosowania danej próby) i przy pomocy funkcji `silliest_generate_proof` dopisujemy do tej próby kolejną linię dowodu i jeżeli próby dowodu po dopisaniu tej linii nie ma w liście prób dowodów dopisujemy ją do niej. Przejdźmy teraz do bardziej szczegółowego omówienia implementacji omawianej funjkcji.

`find_proof_little_smarter` jako argumenty przyjmuje obiekt textttbundle typu `NanoGPTBundle`, który odpowiada za generowanie kolejnych tokenów, zmienną tekstową `prompt` która koduje sekwent do udowodnienia w notacji takiej samej jak w `silliest_generate_proof`, zmienną `temperature` typu `float` która służy do regulacji poziomu determinizmu i losowości podczas generowania kolejnych tokenów, oraz zmienną `max_iter` typu `int`, która określa ile maksymalnie iteracji głównej pętli dopuszczamy.

Na początku funkcja parsuje `prompt` do obiektu `goal` klasy `LittleProblem`. Bazowy kontekst `goal` jest przechowywany w liście `base_context` Następnie tworzymy listę `prompts` w którym umieszczamy oryginalny prompt oraz tabelę `probabilities_wild`, która tworzy listę liczb potrzebnych do obliczenia, prawdopodobieństwa losowania każdego elementu `prompts`, które trzymamy w tabeli `probabilities`. W pętli głównej losujemy element `prompts` i przy pomocy `silliest_generate_proof` generujemy kolejną linię wylosowanej próby dowodu. Tak zaktualizowany prompt zapisujemy w zmiennej `new_prompt_str`. Dopisujemy ją do wylosowanego prompta, a następnie przy pomocy funkcji `extract_proof_part` wyciągamy z tak powstałego prompta do zmiennej `new_proof` część kodującą dowód. Sprawdzamy czy numeracja wierszy w `new_proof` jest poprawna (jeżeli nie jest wyłapujemy wyjątek i przechodzimy do kolejnej iteracji), a następnie parsujemy dowód do zmiennej `parsedProof` przy pomocy funkcji `parseProof` i używając kontekstu `base_context` jako kontekstu bazowego. Jeżeli podczas parsowania otrzymamy błąd wyłapujemy wyjątek i przechodzimy do następnej iteracji pętli. W przeciwnym przypadku jeżeli ostatnia sparsowana linia `parsedProof` jest taka sama jak sekwent docelowy to zwracamy `new_prompt_str`. Jeśli to jeżeli `new_prompt_str` nie ma w tabeli `prompts` to dopisujemy ją do niej, a następnie aktualizujemy prawdopodobieństwa w sposób opisany poniżej.

Do aktualizacji prawdopodobieństw używamy funkcji `prompt_probability_nomalised`. Jako argumenty przyjmuje `bundle` które odpowiada za liczenie prawdopodobieństw kolejnych tokenów, `prompt` którego prawdopodobieństwo analizujemy. Funkcja ta z założenia służy do badania prawdopodobieństwa promptów, które składają się z zakodowania sekwentu, który mamy udowodnić, a także niepustej próby dowodu. Najpierw dzieli `prompt` na część kodującą sekwent oraz część próby dowodu, którą zapisujemy do zmiennej `proof`. Następnie iterując się po kolejnych znakach w `proof` sumujemy do zmiennej `ans_unnormalised` logarytmy prawdopodobieństwo warunkowe jego wystąpienia o ile znaki poprzedzające go w prompcie pozostają bez zmian. Prawdopodobieństwo to obliczamy przy pomocy funkcji `last_token_probability_from_bundle`. Następnie zwracamy `ans_unnormalised` podzielone przez długość części `proof`.

Sama aktualizacja prawdopodobieństw zaczyna się od aktualizacji tabeli `probabilities_wild`, poperzez dopisanie na jej końcu wartości `prompt_probability_normalised` dla używanego `bundle` oraz `new_prompt_str`. (Pole `probabilities_wild[i]` odpowiada polu `prompts[i]`). Dla promptów, które nie posiadają części odpowiadającej za próbę dowodu funkcja `prompt_probability_normalised` nie działa poprawnie toteż nie może zostać użyta do obliczenia wartości `probabilities_wild[0]` odpowiadającego prawdopodobieństwu promptu zawierającego sam problem bez żadnej linii dowodu toteż na wartość tego pola jest ustawiana średnia wartości pozostałych pól tablicy `probabilities_wild`. Ponieważ wartości `probabilities_wild` nie sumują się do 1 i nie są dodatnie do tabeli `probabilities` zapisujemy wartość funkcji `soft_max` obliczonej dla tabeli `probabilities_wild` traktowanej jako matematyczny wektor. Tak zaktualizowanej tabeli `probabilities` używamy do losowania prompta do rozwinięcia w następnej iteracji pętli. Jeżeli po całym wykonaniu pętli wciąż nie znaleźliśmy dowodu funkcja `find_proof_little_smarter` zwraca parę list `prompts` i `probabilities`.

Ostatnia strategia, w której rozbijamy problem na podproblemy zostanie omówiona w następnych rozdziałach.

8 Rozbijanie problemów na podproblemy

Obserwacja 1

W logice klasycznej rachunku zdań:

$$((\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \ldots \quad \wedge \quad \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \varphi)$$

$$\leftrightarrow$$

$$((\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \ldots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \to \Gamma \vdash \varphi)$$

Dowód

$(\to)$

Jeżeli $\Gamma_i \vdash \varphi_i$, to $\Gamma_i, \Gamma \vdash \varphi_i$ ponieważ jesteśmy w stanie skonstruować dowód φ_i w kontekście Γ_i, Γ używając jedynie formuł z kontekstu Γ_i. Tak więc

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \ldots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n)$$

$$\to$$

$$(\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \ldots \quad \wedge \quad \Gamma_n \vdash \varphi_n)$$

$$\to$$

$$\Gamma \vdash \varphi$$

$(\leftarrow)$

Załóżmy nie wprost, że teza nie zachodzi. Wtedy istnieją takie konteksty $\Gamma_1, \Gamma_2, \ldots, \Gamma_n, \Gamma$ i formuły $\phi_1, \phi_2, \ldots, \phi_n$, że $(\Gamma_1, \Gamma \vdash \varphi_1 \wedge \Gamma_2, \Gamma \vdash \varphi_2 \wedge \cdots \wedge \Gamma_n, \Gamma \vdash \varphi_n) \to \Gamma \vdash \phi$ jest prawdziwe oraz, że $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \cdots \wedge \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \phi$ jest fałszywe. Niech

$$F(\Gamma_i) := \text{koninkcja formuł w kontekście } \Gamma_i$$

Z faktu, że w logice klasycznej dla rachunku zdań każde zdanie prawdziwe dla każdego wartościowania jest dowodliwe, to $\Gamma_i \vdash \phi_i$ jest równoważne, z tautologicznością $F(\Gamma_i) \to \varphi_i$.
Fałszywość $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \cdots \wedge \Gamma_n \vdash \varphi_n) \to \Gamma \vdash \phi$ oznacza, że istnieje wartościowanie takie, że dla każdego $i \in \{1, 2, \ldots, n\}$ zachodzi $F(\Gamma_i) \to \phi_i$, ale nie zachodzi $F(\Gamma) \to \phi$, ale to oznacza, że $F(\Gamma)$ dla tego wartościowania zachodzi, z kolei ϕ nie zachodzi. Ale ponieważ dla każdego $i \in \{1, 2, \ldots, n\}$ zachodzi $F(\Gamma_i) \to \phi_i$ i $F(\Gamma)$ zachodzi to znaczy, że $i \in \{1, 2, \ldots, n\}$ zachodzi $(F(\Gamma_i) \wedge F(\Gamma)) \to \phi_i$ i $F(\Gamma)$, czyli zachodzi

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \ldots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \wedge F(\Gamma)$$

, czyli ϕ zachodzi dla tego wartościowania co daje sprzeczność.

Mimo, że dowody w dedukcji naturalnej w notacji Fitcha piszemy zaczynając od przesłanek, a następnie łączymy je odpowiednimi regułami tak by otrzymać jako konkluzję rządaną tezę, to dowody w notacji drzewiastej zwykle generuje się w przeciwnym kierunku. Najpierw w korzeniu drzewa dowodu zapisujemy tezę, którą chcemy udowodnić, a następnie wybieramy regułę i przesłanki do których jeśli zastosujemy wybraną regułę możemy otrzymać szukaną tezę. Następnie próbujemy rekurencyjnie udowodnić owe przesłanki używając ich jako korzenie nowych poddrzew drzewa dowodu. Taka metoda jest szczególnie kożystna gdy formuła jest skomplikowana, i trudno przewidzieć jak zacząć dowód w notacji Fitcha tak by dało się go doprowadzić do końca.

Podobnie jak dla dowodów w notacji Fitcha rozbicia problemów będziemy kodować na dwa sposoby. Z jednej strony ich logiczną strukturę będziemy przechowywać w odpowiedniej klasie, przy pomocy której będziemy kontrolować poprawność rozbić z drugiej będziemy kodować je w odpowiedniej formie tekstowej, tak by do dokonywania rozbić można było wytrenować model językowy typu `nanoGPT` notacji

Klasą która będzie przechowywać logiczną strukturę takiego rozbicia będzie klasa `smashed_problem`, której pola to `problem`, które zawiera obiekt klasy `LittleProblem`, który jest problemem, który rozbijamy, zmienną `subproblems`, która jest listą przesłanek klasy `LittleProblem`, które otrzymujemy po rozbiciu oraz zmienną klasy `Rule`, która jest regułą, przy pomocy której dokonujemy rozbicia.

Gdy mamy obiekt typu `smashed_problem` ważne jest sprawdzić, czy zakodowane w nim rozbicie jest poprawne. To znaczy, czy faktycznie stosując zakodowaną w `Rule` regułę dowodzenia do listy przesłanek w `subproblems` faktycznie możemy otrzymać `Problem` oraz czy owe przesłanki faktycznie są sekwentami tautologicznymi. W tym celu używamy funkcji `is_smashed_correctly`. Funkcja ta jako argument przyjmuje zmienną `SP` typu `smashed_problem`. Na początku funkcja sprawdza czy wszystkie pola w `SP` mają odpowiednie typy i jeżeli tak nie jest wyrzuca ona błąd `ValueError("Bad arguments")`. Następnie dla każdego elementu `SP.subproblems` sprawdza, czy koduje on sekwent tautologiczny poprzez podstawienie każdego możliwego wartościowania zmiennych. Oczywistą wadą tego podejścia jest to, że złożoność tej operacji jest wykładnicza względem liczby różnych zmiennych występujących w danym sekwencie. Należy jednak zaznaczyć, że ze względu na ograniczoność zasobów obliczeniowych dostępnych na potrzeby pracy tworzony model i tak generuje dowody jedynie stosunkowo krótkich formuł. Dla bardziej złożonych formuł możnaby sprawdzenie wszystkich możliwości zastąpić pewnymi heurystykami np sprawdzeniu odpowiednio dużo losowych wartościowań. Jednocześnie samo rozbicie na nietautologiczne sekwenty nie jest bardzo problematyczne, w tym sensie, że zwykle wykonujemy równolegle kilka rozbić i wystarczy, że przynajmniej jedno rozbija problem na sekwenty tautologiczne. Z kolei rozbicia na sekwenty nietautologiczne nie prowadzą w dalszej części do generacji błędnych dowodów, a jedynie sprowadzają niektóre gałęzie generacji dowodu w ślepe uliczki, co spowalnia program, ale nie wpływa na jego poprawność.

Kolejnym krokiem jest sprawdzanie czy reguła `SP.Rule` może faktycznie rozbić `SP.problem` na `SP.subproblems`. Ponieważ nie wszystkie elementy `SP.subproblems` mają ten sam `base_context` co `SP.problem`, uniemożliwia bezpośrednie zastosowanie do nich `SP.Rule.top_down` najpierw tworzymy tabelę `unnormalised_subproblems` której `i`-ty element jest obiektem klasy `LittleProblem`, którego kontekst bazowy to `SP.problem.assumptions.base_context`, z kolei kontekst dodatkowy to wszystkie formuły zawierające się w kontekście bazowym i dodatkowym obiektu `SP.subproblems[i]`, które nie znajdują się w `SP.problem.assumptions.base_context`. Konkluzja `unnormalised_subproblems[i]` jest taka sama jak w `SP.subproblems[i]`. Następnie stosujemy `SP.Rule.top_down` której argumentami są lista obiektów klasy `LittleProblem` o tych samych kontekstach bazowych: `unnormalised_subproblems` i odpowiednio wydedukowane z `SP.problem` pozostałe argumenty. Jeśli wynik tej funkcji jest taki sam jak `SP.problem` to funkcja `is_smashed_correctly` zwraca `True`, zaś w przeciwnym razie `False`.

Format tekstowy zapisu rozbicia formuły jest następujący: w pierwszej lini mamy tekst `Smash: `, po którym zostaje zapisany sekwent który chcemy rozbić. Druga linia zaczyna się od napisu `With: ` po którym następuje nazwa reguły której używamy do rozbicia. Trzecia linia zaczyna się od tekstu `To: ` po którym w następuje pierwsza przesłanka na którą zostaje rozbity rozbijany sekwent. Kolejne sekwenty rozbicia są zapisywane w kolejnych liniach po przecinku. Jeżeli Lista przesłanek jest pusta prompt kodujący rozbicie kończy się po prostu na `To: `. Do sprawdzania czy dany tekst w tym formacie koduje poprawne rozbicie używamy funkcji `check_smash_from_text`, która najpierw parsuje tekst kodujący rozbicie do obiektu typu `smashed_problem`, a następnie sprawdza jego poprawność przy pomocy funjkcji `is_smashed_correctly`. Jeżeli rozbicie jest niepoprawne, lub podany w argumencie prompt nie jest w odpowiednim formacie `check_smash_from_text` wyrzuca błąd `TypeError` z odpowiednią wiadomością. Jeżeli zaś prompt koduje poprawne rozbicie funkcja zwraca zakodowane rozbicie w formie sparsowanej.

Do generowania poprawnych napisów kodujących rozbicie służy funkcja `silliest_generate_smash`. Przyjmuje ona za argumenty zmienną `bundle` typu `NanoGPTBundle` odpoowiadający za generację kolejnych tokenów oraz zmienną tekstową `prompt` kodującą sekwent który chcemy rozbić w omawianym formacie tekstowym. W bloku `try ... exept` funkcja ta generuje kolejne linie rozbicia przy pomocy funkcji `next_line`, które dopisujemy do zmiennej `prompt`. Robimy to aż do momentu gdy tak zaktualizowany `prompt` nie będzie spełniać predykatu `is_ready_to_check_smash`, który zwraca prawdę wtedy i tylko wtedy gdy podana jako argument zmienna tekstowa `prompt`, po uprzednim usunięciu znaków nowej linii z jej końca ma nie mniej niż 3 linie i jej ostatnim znakiem nie jest `,`. Kiedy ten predykat bę-

dzie spełniony sprawdzamy najpierw przy pomocy poprawność rozbicia zakodowanego w `prompt` przy pomocy `check_smash_from_text` i jeśli nie jest poprawne wyrzucamy odpowiedni błąd, z kolei jeśli jest poprawne sprawdzamy czy w przesłankach na które rozbiliśmy rozbijany sekwent nie pojawiają się zmienne inne niż w rozbijanym sekwencie. Jeżeli tak się dzieje zwracamy wyrzucamy błęd. Jeśli podczas rozbijania zostanie wyłapany błąd zostawiamy w prompcie tylko pierwszą linię i znak nowej linii, a następnie powtarzamy całą procedurę, aż do momentu kiedy nie wygenerujemy poprawnego rozbicia.

W prototypowych wersjach programu rozważane były wyłącznie dowody formuł tautologicznych, a więc sekwentów o pustym kontekście. Ograniczenie to okazało się jednak niewystarczające, ponieważ podczas rozbijania celu na prostsze podproblemy pojawiają się sekwenty o niepustych kontekstach. Aby system mógł poprawnie obsługiwać takie przypadki, konieczne było rozszerzenie go o możliwość dowodzenia sekwentów z założeniami.

Z Obserwacji 1 wynika przez prostą indukcję po drzewie dowodu, że dowód $\Gamma \vdash \varphi$ w którym kontekst każdego wierzchołka zawiera Γ, istnieje wtedy i tylko wtedy gdy istnieje równinież dowód sekwentu zawierający w liściach sekwenty o pustym kontekście. Toteż bez straty ogólności w całym dowodzie $\Gamma \vdash \varphi$ możemy przyjąć za prawdziwe założenia z kontekstu `Gamma`. Jest to użyteczne ponieważ w procesie generowania dowodu mamy gwarancję, że nie trafimy w ślepą uliczkę w postaci odrzuceniu niektórych założeń z dowodzonego sekwentu, do których nasz model będzie miał trudność wrócić. Jest to motywacja dlaczego `Context` jest podzielony na dwie tabele `base_context` i `additional_context`.

9 Klasa `node`

Aby połączyć zarówno rozbijanie problemów na podproblemy oraz generowanie dowodów w notacji Fitcha w jeden kompleksowy system będę używał klasy `node`.

`node` koduje drzewo przeszukiwań dowodu. Drzewo to ma wierzchołki dwóch rodzajów. Pierwszy z nich w polu `Interior` zawiera obiekt klasy `LittleProblem` z kolei drugi zawiera obiekt klasy `smashed_problem`. Dla uproszczenia będę wierzchołki pierwszego rodzaju nazywać `LPNode`, z kolei wierzchołki drugiego rodzaju będę nazywać `SPNode` mimo, że takie nazewnictwo nie występuje w samym kodzie programu. Dzieci rodzaju `LPNode` są rodzaju `SPNode`, z kolei dzieci rodzaju `SPNode` są rodzaju `LPNode`. Precyzyjniej dziećmi wierzchołków zawierających w polu `Interior` obiekt klasy `LittleProblem` kodujący pewien sekwent, w polu `Interior` mają obiekty klasy `smashed_problem` kodujące poprawne rozbicia tego obiektu. Z kolei dzieci wierzchołków zawierających w polu `Interior` obiekt klasy `smashed_problem` mają w polu `Interior` obiekty klasy `LittleProblem` zawierające przesłanki tego rozbicia.

Poza zmienną `Interior` klasa `node` zawiera też pole `Parent` które wskazuje na rodzica wierzchołka, pole `Children` listę dzieci, pole `Text`, które dla `LPNode` zawiera prompta z zapytaniem o dowód sekwentu w polu `Interior` lub zapytanie wraz z pełnym dowodem oraz pole `Probability`, które dla `SPNode` określa prawdopodobieństwo, że wybierzemy do analizy ten wierzchołek schodząc w dół od jego rodzica. Ostatnim polem jest zmienna boolowska `Value`. Dla liści typu `LittleProblem` ma wartość `False` jeśli w polu `Text` zawiera ona prompta jedynie z zapytaniem o dowód sekwentu i `True` jeśli w tym polu prompt składa się z zarówno zapytania jak i dowodu. Dla wierzchołków wewnętrznych rodzaju `LPNode` wartość `Value` to alternatywa wartości `Value` dzieci tego wierzchołka. Wierzchołek rodzaju `SPNode` może być liściem tylko wtedy, gdy lista przesłanek obiektu klasy `smashed_problem` w zakodowanym w jego polu `Interior` jest pusta. Wtedy też wartość pola `Value` dla tego wierzchołka jest równa `True`. Dla wierzchołków wewnętrznych rodzaju `SPNode` pole `Value` jest równe koniunkcji wartości pól `Value` jego dzieci.

Metoda `Smash_Leaf` klasy `node` odpowiada za doczepianie do dzieci rodzaju `SPNode` które zawierają zakodowane rozbicia pola `Interior` swojego rodzica, a następnie doczepienia do tych dzieci ich dzieci rodzaju `LPNode` z przesłankami rozbicia kodowanego w ich rodzicach. Funkcja ta przyjmuje jako argumenty obiekt `bundle_smashing` typu `NanoGPTBundle` odpowiadający za generowanie rozbić, oraz zmienna całkowitoliczbowa `deg` oznaczająca ile rozbić (nie koniecznie różnych) które będziemy wykonywać. Na początku tworzymy prompta zaczynającego się od `Smash:` , po którym następuje zapis sekwentu, który chcemy rozbijać i znak nowej lini. Następnie `deg` rzay przy pomocy metody `silliest_generate_smash` dopisujemy do tego prompta tekstową reprezentację rozbicia. Ponieważ częso model umie dość dobrze znajdować najkożystniejsze rozbicie, często one się powtażają. Dlatego tekstowe reprezentacje wygenerowanych rozbić przechowujemy jako klucze słwonika `weights`, w którym

weights[i] jest równa liczbie ile razy uruchomiona funkcja Smash_Leaf wygenerowała i. Następnie do tabeli Children rozbijanego wierzchołka dodajemy obiekty klasy node, które w polu Interior mają zparsowaną przy pomocy check_smash_from_text do obiektu klasy smashed_problem wygenerowaną wcześniej tekstową reprezentację rozbicia, z kolei w polu Probability ma iloraz liczby ile razy dana reprezentacja tekstowa została wygenerowana i liczby deg. Kolejnym krokiem jest dla każdego z tych dopisanych dzieci dopisanie do tabeli Children ich dzieci obiekty klasy node, które w polu Interior mają obiekty typu LittleProblem reprezentujące przesłanki rozbić zakodowanych w ich rodzicach. Załóżmy, że któreś z dzieci wierzchołka podanego jako argument dla funkcji Smash_Leaf po wykonaniu wcześniej omawianych kroków nie ma dzieci. Oznacza to, że sekwent którego rozbicie jest zakodowane w tym dziecku nie potrzebuje dodatkowych kroków dowodu by go udowodnić. Ustawiamy wtedy pole Value tego dziecka na True i aktualizujemy wartości pola Value dla wszystkich jego przodków.

Kolejną metodą klasy node jest funkcja RunLeaf. Próbuje najpierw znaleźć dowód sekwentu, który w polu Interior przechowuje obiekt, który ją wywołuje, a jeśli nie jest w stanie tego zrobić dokonuje jego rozbicia przy pomocy metody Smash_Leaf. Funkcja ta przyjmuje dwa obiekty klasy NanoGPTBoundle. Pierwszym z nich jest obiekt bundle_proving generujący dowody sekwentów, drugim zaś z kolei jest bundle_smashing odpowiadający za generowanie rozbić. Kolejnym argumentem jest zmienna zmiennoprzecinkowa temperature regulująca poziom determinizmu przy generowaniu dowodów, zmienna całkowitoliczbowa max_iter, która określa maksymalną liczbę iteracji prób dopisania kolejnej linii przy próbie generowania dowodu, a także deg, która reguluje ile rozbić (niekoniecznie różnych) mamy wykonać, jeżeli sekwentu nie uda się udowodnić. Na początku funkcja sprawdza czy wywołujący ją obiekt jest liściem i czy jest LPNode. Jeżeli tak nie jest funkcja zwraca wyjątek z odpowiednim komentarzem. Następnie tworzymy prompta z zapytaniem o dowód sekwentu, kóry znajduje się w polu Interior wierzchołka w którym się znajdujemy. Zapytanie to podajemy do funkcji find_proof_little_smarter która przy pomocy bundle_proving próbuje udowodnić zadany w prompcie sekwent. Jeżeli to się uda funkcja ta zwraca obiekt klasy str z dowodem, jeżeli zaś nie zwraca ona tabelę prób i pomocniczych prawdopodobieństw. Sprawdzamy więc czy find_proof_little_smarter zwróci zmienną typu str a jeśli tak to w polu Text wierzchołka w którym się znajdujemy zapisujemy ten dowód, a następnie ustawiamy wartość jego pola Value na True i aktualizujemy wartości pola Value dla jego przodków. W przeciwnym razie rozbijamy obiekt którym wywołaliśmy metodę RunLeaf metodą Smash_Leaf używając bundle_smashing i podaną w argumencie do RunLeaf zmienną deg. Jeżeli RunLeaf uda się znaleźć dowód zwracamy True. W przeciwnym wypadku zwracamy False.

Kolejną metodą klasy node jest funkcja TryNode. Funkcja ta próbuje przeprowadzić cały dowód zadanego sekwentu a także jeśli to się nie uda zaktualizować drzewo tak by było bliższe znalezienia owego dowodu. Argumenty tej funkcji mają takie same nazwy, typy oraz zastosowania jak argumenty funkcji RunLeaf. Jeżeli znalezienie dowodu się powiedzie funkcja zwraca True, a w przeciwnym wypadku zwraca False. Działanie tej funkcji dzielimy na trzy nietrywialne przypadki przy czym przez znalezienie dowodu dla każdego z nich rozumiemy następująco. Dla przypadku liścia rodzaju LPNode jest to udowodnienie sekwentu zakodowanego w polu Interior, dla wierzchołków wewnętrznych rodzaju LPNode przez znalezienie dowodu rozumiemy znalezienie dowodu dla któregoś z jego dzieci, z kolei dla wierzchołków wewnętrznych rodzaju SPNode przez znalezienie dowodów dla wszystkich jego dzieci. Ostatnim trywialnym przypadkiem jest gdy wierzchołek wywołujący funkcję ma w polu Value wartość True, co oznacza, że dowód dla tego wierzchołka już został znaleziony i wtedy funkcja TryNode. zwraca True. Jeżeli obiekt wywołujący tą funkcję jest liściem typu LPNode uruchamiamy dla dla niego metody RunLeaf, z pozostałymi argumentami takimi samymi jak te podane jako argumenty dla TryNode i zwracamy jej wartość. Jeżeli jesteśmy w wierzchołku wewnętrznym rodzaju LPNode losujemy jego dziecko, króre jest rodzaju SPNode i rekurencyjnie wykonujemy TryNode wywołane przez to dziecko z pozostałymi argumentami takimi samymi jak jak w wywołaniu dla jego rodzica i zwracamy wartość tego wywołania. Jeżeli zaś jesteśmy w wierzchołku SPNode wywołujemy funkcję TryNode dla kolejnych jego dzieci (pozostałe argumenty dla każdego dziecka są takie same jak dla wywołania dla rodzica), aż do momentu kiedy któreś z tych wywołań zwróci False. Wtedy wywołanie dla rodzica tych dzieci również zwraca False. Jeżeli każde wywołanie dla dzieci zwróci True to funkcja dla rodzica również zwraca True.